

CM10 : Arbres de décision et K-moyennes

1 Arbres de décision (et de régression)

1.1 Fonctionnement

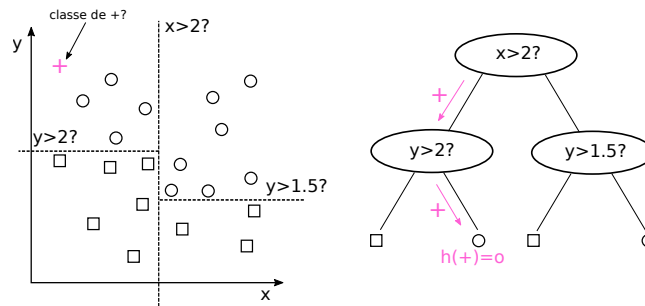
Les arbres de décisions - nous verrons les arbres de régression par la suite - sont une méthode de classification présentant plusieurs avantages :

- Interprétable
- Très rapide et efficace (notamment la version random forest)
- Peu s'utiliser sur des données non linéairement séparables

L'idée est de construire un arbre de questions successives. Chaque question porte sur une feature et un seuil, c'est à dire :

La donnée $\vec{x}$ a-t-elle une valeur plus grande que s pour la feature d ? Ou de façon équivalente : $x_d > s$?

Suivant la réponse à cette question on branche dans une partie ou l'autre de l'arbre, chaque sous-partie a des questions différentes. On va ainsi de question en question descendre dans l'arbre jusqu'à atteindre une feuille qui correspondra à une région de nos données. On regarde alors la classe de nos données d'entraînement pour cette région, la classe majoritaire est alors la prédiction pour $\vec{x}$.



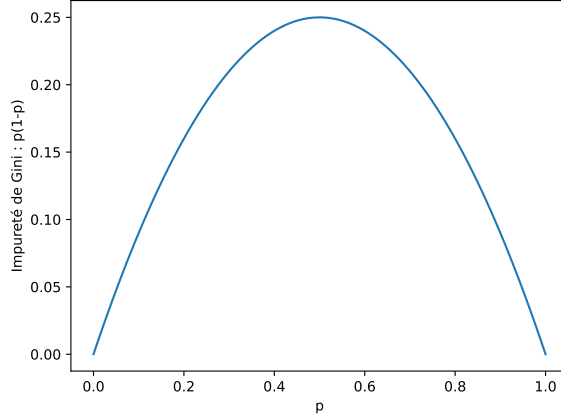


Figure 1: Classification binaire, l'impureté de Gini atteint son maximum pour $p = 0.5$.

1.2 Construction de l'arbre

La question est maintenant de savoir comment construire l'arbre.

Intuitivement on aimerait limiter le nombre de découpes (ce qui évite l'overfitting) et que chaque découpe sépare au mieux les classes en deux ensembles distincts. Pour nous aider à formaliser cette intuition, nous allons définir la pureté d'un ensemble de points.

1.2.1 Impureté de Gini

Soit un ensemble de points $S = \{(x_1, y_1), (x_2, y_2) \dots (x_n, y_n)\}$ avec $y_i \in \{1 \dots c\}$, où c est le nombre de classes. Soit $S_k \subset S$ l'ensemble des points de S de classe k , c'est à dire $S_k = \{(x, y) \in S | y = k\}$.

On définit p_k la probabilité de tirer un point appartenant à k dans S :

$$p_k = \frac{|S_k|}{|S|}.$$

L'impureté de Gini de S est alors définie par :

$$G(S) = \sum_k^K p_k \cdot (1 - p_k)$$

Intuitivement, l'impureté de d'un ensemble est élevée si l'ensemble contient à peu près le même nombre d'éléments de chaque classe, elle est même maximale si il y exactement autant d'éléments de chaque classe. A l'inverse, l'impureté est faible si l'ensemble contient principalement des éléments d'une même classe.

Quelle est l'impureté d'un ensemble dont tous les éléments sont de la même classe ?

1.2.2 Algorithme

L'algorithme procède de façon *greedy*, à chaque étape on cherche la découpe qui minimise la somme de l'impureté des deux nouveaux ensembles obtenus. Pour un ensemble de N points dimension D , combien il y a t il de découpes possible ?

Il est toujours possible de continuer à découper l'espace jusqu'à obtenir une impureté de 0 sur l'ensemble d'entraînement, le problème est que cela peut très facilement conduire à de l'overfitting. Pour limiter cela on peut décider d'arrêter l'algorithme après k découpes consécutives, même si les ensembles obtenus ont une impureté non nulle. k est alors un hyperparamètre de l'algorithme, qui correspond à la **profondeur de l'arbre**.

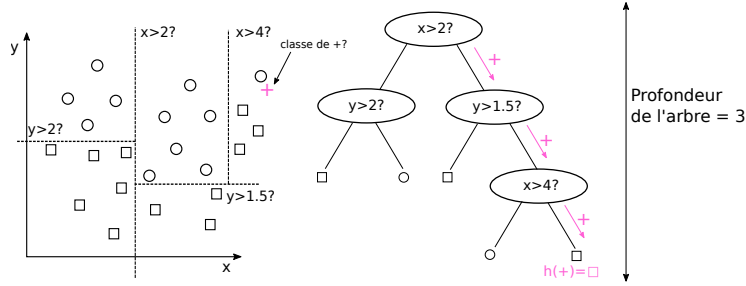


Figure 2: Arbre de décision de profondeur 3. Toutes les feuilles ne sont pas pures. Il est possible d'augmenter la profondeur de l'arbre pour augmenter la pureté ou le conserver tel quel pour éviter l'overfitting.

1.3 Arbres de régression

Pour les arbres de régression, la prédiction pour une feuille de l'arbre est simplement la moyenne des valeurs des données d'entraînement correspondant à cette feuille.

Pour la construction de l'arbre c'est pareil, il nous faut simplement une nouvelle définition de l'impureté. Pour cela, on choisit simplement l'erreur quadratique moyenne. C'est à dire pour un ensemble de point $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$:

$$L(S) = \sum_k^K (y_k - \bar{y}_S)^2$$

Avec $\bar{y}_S$ la moyenne des labels des points dans S :

$$\bar{y}_S = \frac{1}{K} \sum_k^K y_k$$

2 K-moyennes

2.1 Description

L'algorithme des k-moyennes est un algorithme **non-supervisé**. L'input de l'algorithme est un ensemble de points $\mathcal{D} = \{(x_1, x_2, \dots, x_N)\}$ non étiquetés (sans labels y_i). Nous avons déjà vu un tel algorithme, lequel ?

L'objectif est ici de trouver des groupes de points (clusters) tel que les points dans chaque groupe soient "proches" les uns des autres. Combien de groupes souhaite-t-on trouver ? C'est l'hyperparamètre K des K-moyennes, c'est donc un input de l'algorithme.

L'idée de l'algorithme est simple et itérative. On part de K "centres" placés aléatoirement dans l'espace des données, appelés **centroïdes**. On répète ensuite deux étapes jusqu'à convergence :

Étape 1 : Assignment. Chaque point est assigné au centroïde le plus proche. On obtient ainsi K groupes.

Étape 2 : Mise à jour. On recalcule la position de chaque centroïde comme la moyenne des points qui lui ont été assignés. Les centroïdes se "déplacent" vers le centre de gravité de leur groupe.

On répète ces deux étapes jusqu'à ce que les centroïdes ne bougent plus, l'algorithme a convergé.

2.2 Algorithme

Formellement, on cherche à minimiser la distance de chaque point au centroïde de la classe qui lui est assignée, c'est à dire au centroïde le plus proche. La fonction que l'on cherche à minimiser est donc :

$$J = \sum_n^N \sum_k^K a_{n,k} \|\vec{x}_n - \vec{\mu}_k\|_2^2$$

Avec,

$$\text{Position d'un centre : } \vec{\mu}_k = \frac{\sum_n^N a_{n,k} \vec{x}_n}{\sum_n^N a_{n,k}}$$

$$\text{Affectation : } a_{n,k} = \begin{cases} 1 & \text{si } k = \operatorname{argmin}_i \|\vec{x}_n - \vec{\mu}_i\|_2 \\ 0 & \text{sinon} \end{cases}$$

Si l'on reprend notre description précédente de l'algorithme :

Étape 1 : À affectations fixées, on minimise J (en ré-assignant les représentants)

Étape 2 : À représentants fixés, on minimise J (en ré-assignant les affectations)

2.2.1 Minimisation par rapport à la position des centres

$$J = \sum_n^N \sum_k^K a_{n,k} \|\vec{x}_n - \vec{\mu}_k\|_2^2 = \sum_n^N \sum_k^K a_{n,k} (\vec{x}_n - \vec{\mu}_k)^2$$

On cherche à minimiser J en modifiant la position des centres, on dérive donc selon μ .

$$\frac{\partial J}{\partial \vec{\mu}_j} = \sum_n^N -a_{n,j} \cdot 2(\vec{x}_n - \vec{\mu}_j)$$

On cherche un minimum, on regarde donc où la dérivée s'annule.

$$\begin{aligned} \frac{\partial J}{\partial \vec{\mu}_j} &= 0 \\ \Leftrightarrow \sum_n^N -a_{n,j} \cdot 2(\vec{x}_n - \vec{\mu}_j) &= 0 \\ \Leftrightarrow \sum_n^N a_{n,j} \cdot \vec{\mu}_j - \sum_n^N a_{n,j} \cdot \vec{x}_n &= 0 \\ \Leftrightarrow \sum_n^N a_{n,j} \cdot \vec{\mu}_j &= \sum_n^N a_{n,j} \cdot \vec{x}_n \\ \Leftrightarrow \vec{\mu}_j &= \frac{1}{\sum_n^N a_{n,j}} \sum_n^N a_{n,j} \cdot \vec{x}_n \end{aligned}$$

On retrouve bien notre règle de mise à jour des centres.

2.2.2 Minimisation par rapport à l'affectation des points

Cette étape ne nécessite pas de calcul différentiel : J se décompose en N termes indépendants, et chacun est minimisé par un simple choix discret.

$$\begin{aligned} a_{n,k} &\in 0, 1, \forall n \sum_k^K a_{n,k} = 1 \\ J &= \sum_n^N \sum_k^K a_{n,k} \cdot \|\vec{x}_n - \vec{\mu}_k\|_2^2 \\ J &= \sum_n^N J_n \quad J_n = \sum_k^K a_{n,k} \cdot \|\vec{x}_n - \vec{\mu}_k\|_2^2 \\ J_n &= \|\vec{x}_n - \vec{\mu}_{k^*}\|_2^2 \text{ pour } a_{n,k^*} = 1 \end{aligned}$$

On veut donc,

$$k^* = \operatorname{argmin}_k \|\vec{x}_n - \vec{\mu}_k\|_2^2$$

$$a_{n,k} = 1 \text{ si } k = k^*, 0 \text{ sinon}$$